Mawlana Bhashani Science and Technology University
Santosh, Tangail-1902, Bangladesh
Department of Information and Communication Technology

Smart Attendance System

Automatic class attendance using face recognition

Submitted by
Rathul Hasan Saion
ID: IT-24037

November 2026

Contents

1 Introduction **1**

1.1 Background 1
1.2 Motivation 1
1.3 Scope of the Project 1
1.4 Organization of the Report 1

2 Problem Statement **3**

3 Objectives **4**

4 Literature Review **5**

4.1 Evolution of Attendance Systems 5
4.2 Face Recognition Pipeline 5
4.3 Research Gap 5

5 Proposed System **6**

5.1 System Overview 6
5.2 Main Features 6
5.3 User Roles 6
5.4 Working Procedure 7
5.5 Use Case Diagram 7
5.6 Sequence Diagram 7
5.7 Data Flow Diagrams 7
5.8 Module Structure 7
5.9 User Interface Design 8

6 Methodology **13**

6.1 Development Model 13
6.2 Project Timeline 13
6.3 Face Recognition Method 13
6.4 Database Design 14
6.4.1 Life Cycle of a Class Session 15
6.5 Technologies Used 15
6.6 Security Measures 15

7 Requirements **17**
7.1 Functional Requirements 17
7.2 Non-Functional Requirements 17
7.3 Hardware and Software Requirements 17

8 Expected Outcome **19**
8.1 Limitations 19
8.2 Future Work 19

9 Conclusion **20**

10 How to Run **21**

List of Figures

5.1 System architecture of the Smart Attendance System 6
5.2 Flowchart of taking attendance 9
5.3 Use case diagram 10
5.4 Sequence diagram of taking attendance 10
5.5 Data flow diagram, level 0 (context diagram) 11
5.6 Data flow diagram, level 1 11
5.7 Layered module structure 12
5.8 Wireframes of the main pages 12

6.1 Waterfall model used in this project 13
6.2 Gantt chart of the project work plan 14
6.3 Enrolment of a student with face photographs 14
6.4 Face recognition pipeline used during a class 15
6.5 Entity-relationship diagram 15
6.6 State diagram of a class session 16

List of Tables

2.1 Comparison of attendance methods 3

5.1 Permissions of the two roles 7

6.1 Technologies used in the project 16

7.1 Hardware and software requirements 18

1. Introduction

1.1 Background

Taking attendance is one of the most routine activities in a university classroom. In most departments it is still done on paper: the teacher calls each name or passes a sheet around, and the marks are copied later into a monthly record. This process consumes lecture time, invites mistakes and allows proxy attendance, where one student marks an absent classmate as present.

The **Smart Attendance System** is a web-based application that marks the attendance of a whole class from a single photograph. The system finds the faces in the photograph, converts each face into a numerical embedding, compares it with the embeddings of the students enrolled in the selected course, and marks the recognised students as present. The teacher then reviews the result, corrects any mistake and finalizes the class. Session reports and month sheets are produced automatically.

1.2 Motivation

A typical class in our department has many students and only a short lecture period. Even two or three minutes spent on roll calling in every class add up to hours of lost teaching time over a semester. At the same time, attendance often decides whether a student may sit in the final examination, so the record must be accurate and difficult to tamper with. These two needs, saving time and improving trust in the record, motivated this project.

1.3 Scope of the Project

The system is intended for use inside a department. It covers the management of students, courses, teachers and enrolments, the capture and storage of face data, the recognition of students from a class photograph, the review and correction of results, and the production of reports. It runs as a web application on a local server so that teachers can use an ordinary browser on a laptop or phone. Hardware such as door-mounted cameras and large-scale deployment across a campus are outside the scope of this work.

1.4 Organization of the Report

Chapter 2 states the problem. Chapter 3 lists the objectives. Chapter 4 reviews related work. Chapter 5 describes the proposed system with its architecture, flowchart, sequence diagram, data flow diagrams and user interface. Chapter 6 describes the methodology, the project timeline, the face recognition method and the database design. Chapter 7 lists the requirements,

Chapter 8 gives the expected outcome and Chapter 9 concludes. Chapter 10 explains how to run the project.

2. Problem Statement

Manual attendance has the following problems.

- **Loss of lecture time.** Roll calling or circulating a sheet takes minutes of every class.
- **Proxy attendance.** A present student can mark an absent classmate and there is no simple way to verify identity.
- **Human error.** Calculating totals, percentages and month sheets by hand is slow and error prone.
- **Poor record keeping.** Paper registers can be lost, damaged or altered, and old records are hard to search.
- **Lack of access control.** In a shared system any user can delete important records by mistake unless permissions are separated by role.

Table 2.1 compares the common approaches to attendance with the proposed one.

Table 2.1: Comparison of attendance methods

Method	Speed	Proxy risk	Extra device	Automatic report
Paper register	Slow	High	None	No
Card / RFID	Fast	High (cards can be lent)	Reader	Yes
Fingerprint	Slow for a class	Low	Scanner	Yes
QR code	Fast	Medium (code can be shared)	Phone	Yes
Face recognition	**Fast, whole class at once**	**Low**	**Camera only**	**Yes**

3. Objectives

The main objective is to build a web-based system that records class attendance automatically using face recognition. The specific objectives are:

1. To manage students, courses and enrolments through a simple web interface.
2. To store several reference photographs and face embeddings for every student.
3. To recognise many students from one class photograph and mark them present.
4. To let the teacher review unknown faces and correct marks before finalizing a class.
5. To prepare session reports and month-wise attendance sheets.
6. To provide two roles, teacher and administrator, with the administrator alone allowed to delete students and courses.

4. Literature Review

4.1 Evolution of Attendance Systems

Attendance systems have moved from paper registers to card and RFID readers, fingerprint scanners, QR-code applications and face recognition. Card and RFID systems identify the card rather than the person, so cards can be lent. Fingerprint scanners are reliable but slow for large classes and need dedicated devices. QR-code applications depend on the student's phone and can be shared. Face recognition identifies the person, needs only a camera, and can process many people in one image.

4.2 Face Recognition Pipeline

A face recognition pipeline has three stages: face detection, feature extraction (embedding) and matching.

Detection. A detector such as RetinaFace [3] locates every face in the image and returns a bounding box and facial landmarks.

Embedding. The detected face is aligned and passed through a deep network that maps it to a vector in which faces of the same person lie close together. FaceNet [1] and ArcFace [2] are well-known examples, and the InsightFace library [4] provides ready-made implementations.

Matching. Vectors are compared with cosine similarity and the best match above a threshold is accepted. A face whose best score stays below the threshold is treated as unknown.

4.3 Research Gap

Existing face-based attendance projects often stop at a desktop script. They seldom include course management, teacher review of uncertain results, user roles and reports. This project combines recognition with a complete web workflow so that it can be used in a real classroom.

5. Proposed System

5.1 System Overview

The teacher uses a web browser to open the application, which runs on the department network. The Flask server [5] handles login, roles and the attendance workflow, calls the face service for recognition, and stores data in an SQLite database [6] and a photo folder. The architecture is shown in Figure 5.1.

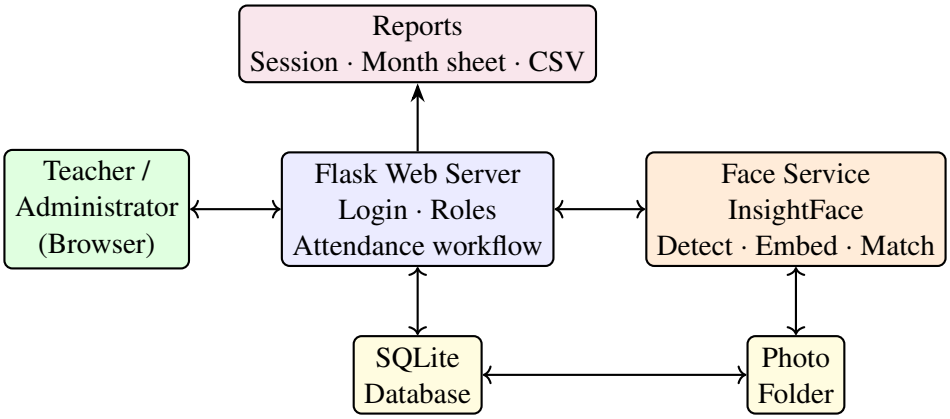


Figure 5.1: System architecture of the Smart Attendance System

5.2 Main Features

- Student, course and teacher management with edit and delete (delete for the administrator only)
- Several face photographs per student, stored as embeddings
- Taking attendance from a camera frame or an uploaded class photograph
- Review of unknown faces and manual correction of marks
- Finalizing a class, and re-opening it if a correction is needed
- Session report, month sheet and CSV export
- Hashed passwords, login protection and a dark mode option

5.3 User Roles

Table 5.1 shows what each role can do.

Table 5.1: Permissions of the two roles

Action	Teacher	Administrator
Log in and change own password	Yes	Yes
Add / edit students and courses	Yes	Yes
Take and review attendance	Yes	Yes
Finalize or re-open a class	Yes	Yes
View reports and export CSV	Yes	Yes
Manage teacher accounts	No	Yes
Delete students and courses	No	Yes

5.4 Working Procedure

The teacher selects a course and a date and provides a class photograph. The system detects the faces and compares them with the students enrolled in that course. Students above the threshold are marked present, the remaining faces go to an unknown list, and the teacher reviews the result. When the class is finalized, unmarked students become absent. The flow is shown in Figure 5.2.

5.5 Use Case Diagram

Figure 5.3 shows the main interactions of the two actors with the system.

5.6 Sequence Diagram

Figure 5.4 shows the order of messages between the teacher, the web server, the face service and the database while one class is taken.

5.7 Data Flow Diagrams

The context diagram in Figure 5.5 shows the system as one process with the two external users. Figure 5.6 breaks it into five processes and three data stores.

5.8 Module Structure

The program is divided into four layers (Figure 5.7). Each layer talks only to the layer below it, so the face service can be replaced without changing the web pages.

5.9 User Interface Design

Figure 5.8 shows the wireframes of the login page and of the attendance review page, which is the page the teacher uses most.

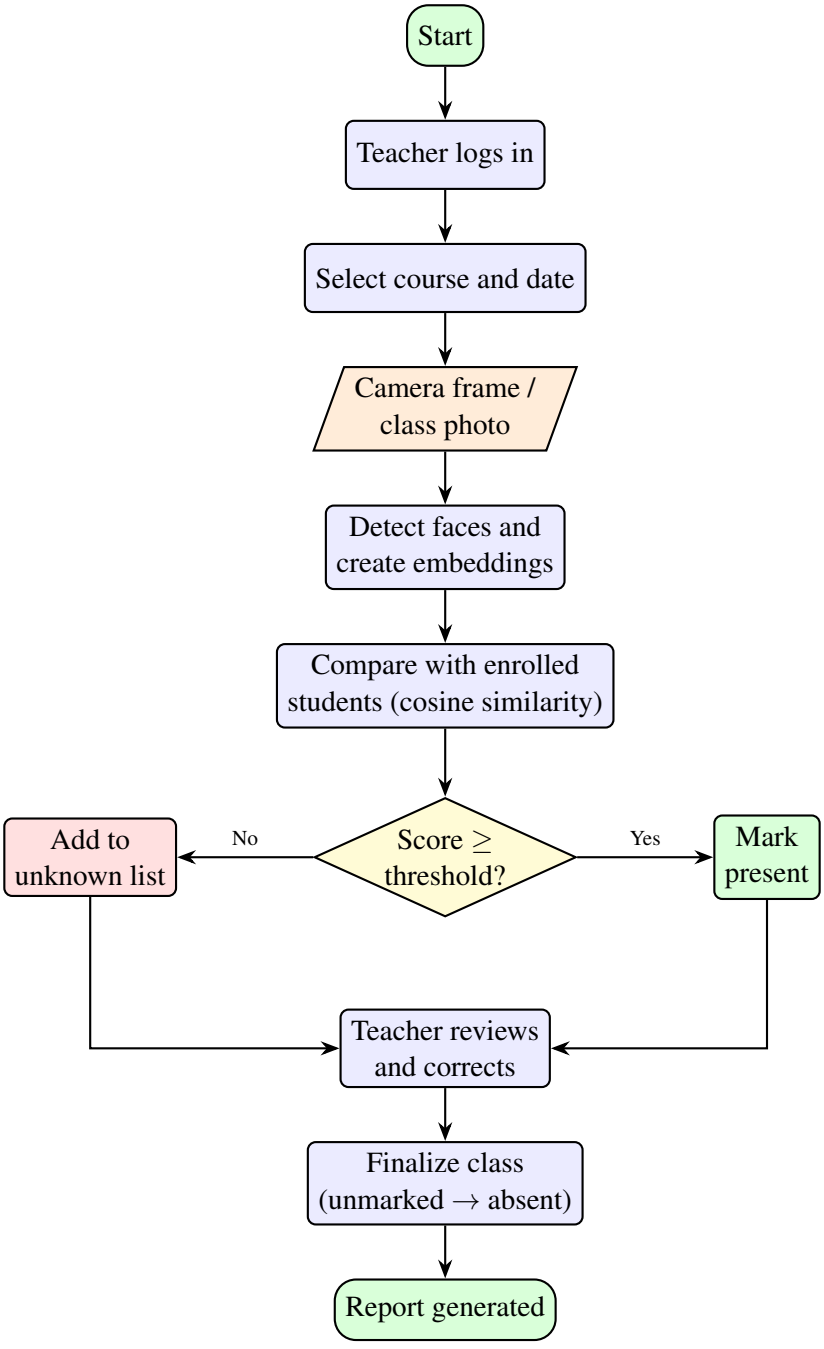


Figure 5.2: Flowchart of taking attendance

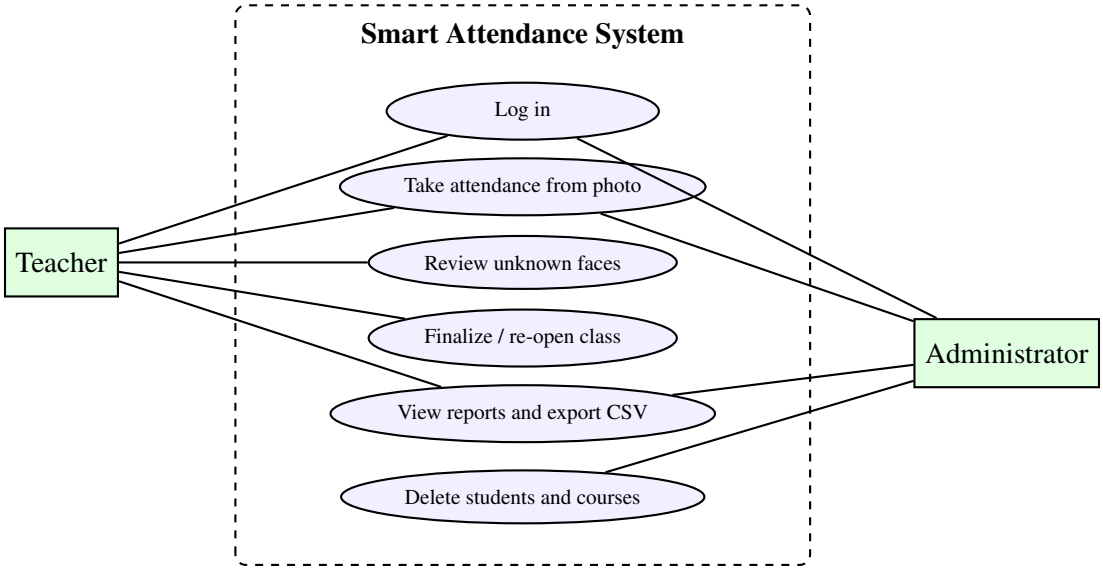


Figure 5.3: Use case diagram

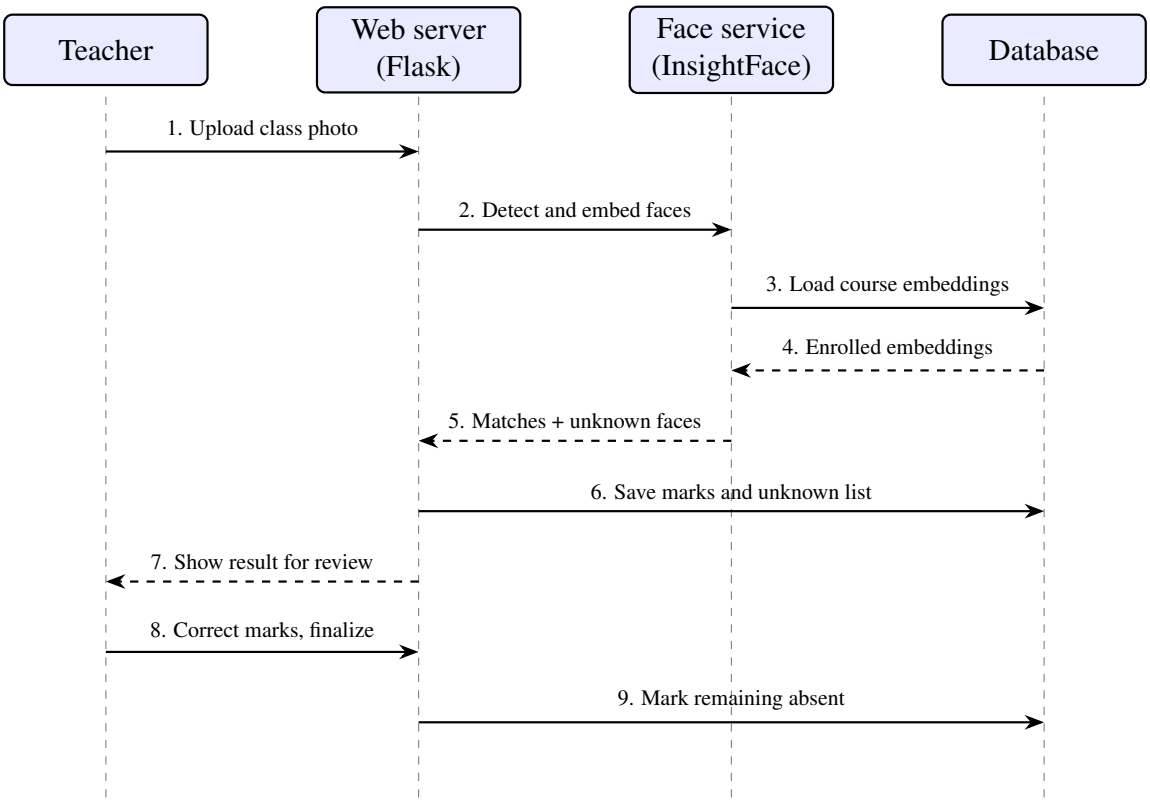


Figure 5.4: Sequence diagram of taking attendance

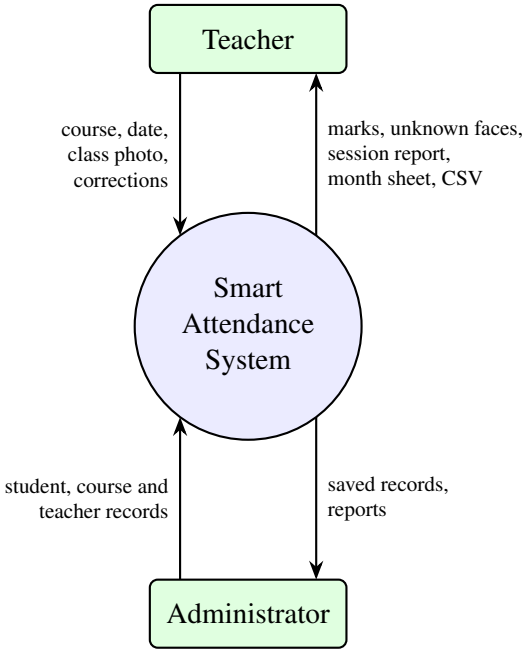


Figure 5.5: Data flow diagram, level 0 (context diagram)

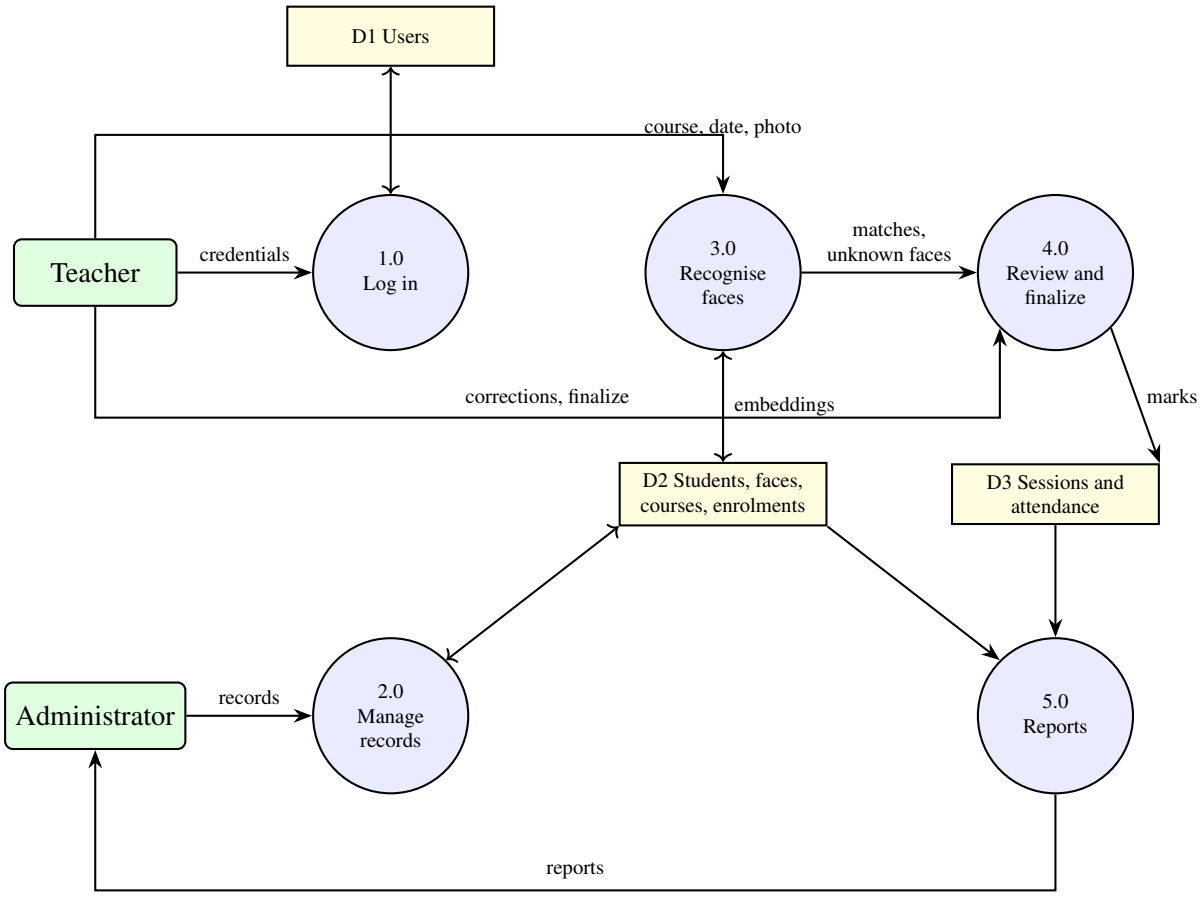


Figure 5.6: Data flow diagram, level 1

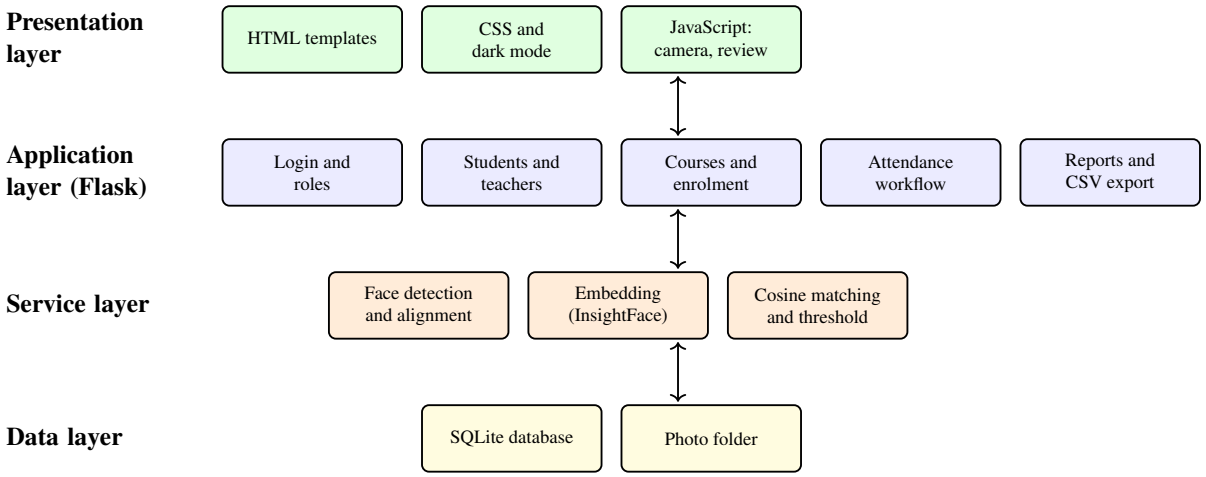
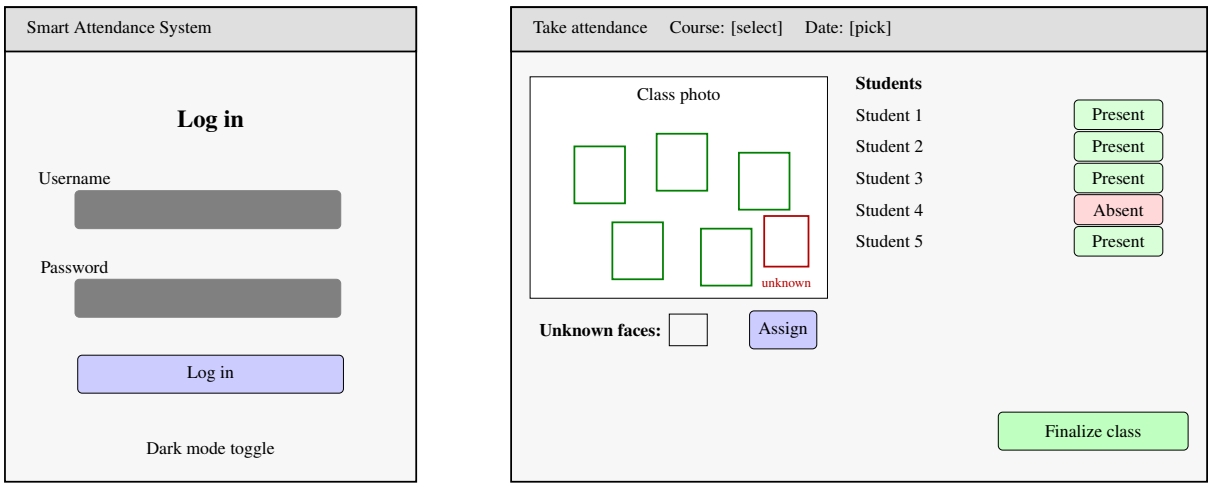


Figure 5.7: Layered module structure



(a) Login page

(b) Attendance review page

Figure 5.8: Wireframes of the main pages

6. Methodology

6.1 Development Model

The project follows the **waterfall model**. Each phase is completed and reviewed before the next begins, which suits this project because its requirements are clear from the start. The phases are requirement analysis, design, implementation, testing and maintenance (Figure 6.1).

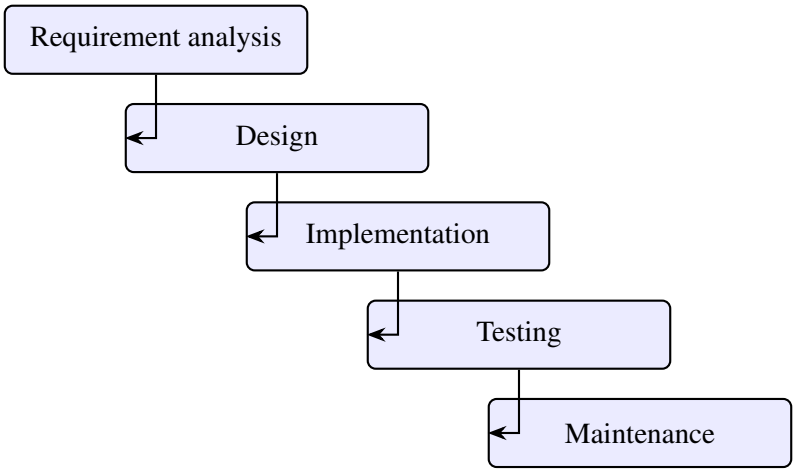


Figure 6.1: Waterfall model used in this project

6.2 Project Timeline

The work is planned over twelve weeks. Figure 6.2 shows the Gantt chart. Documentation runs alongside implementation and testing so that the report is ready when the system is.

6.3 Face Recognition Method

For every enrolled student several reference photographs are stored. Each photograph is processed by the detector and the embedding network and the resulting vector is saved in the database. During attendance, every face found in the class photograph is converted into a vector $\mathbf{q}$ and compared with the stored vectors $\mathbf{e}_i$ of the students enrolled in the selected course using cosine similarity:

$$s_i = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \, \|\mathbf{e}_i\|}. \tag{6.1}$$

The student with the highest score is the candidate. If $\max_i s_i \geq \tau$, where τ is the threshold, the student is marked present; otherwise the face is placed in the unknown list for the teacher to review. Comparing only with students enrolled in the selected course reduces the number of

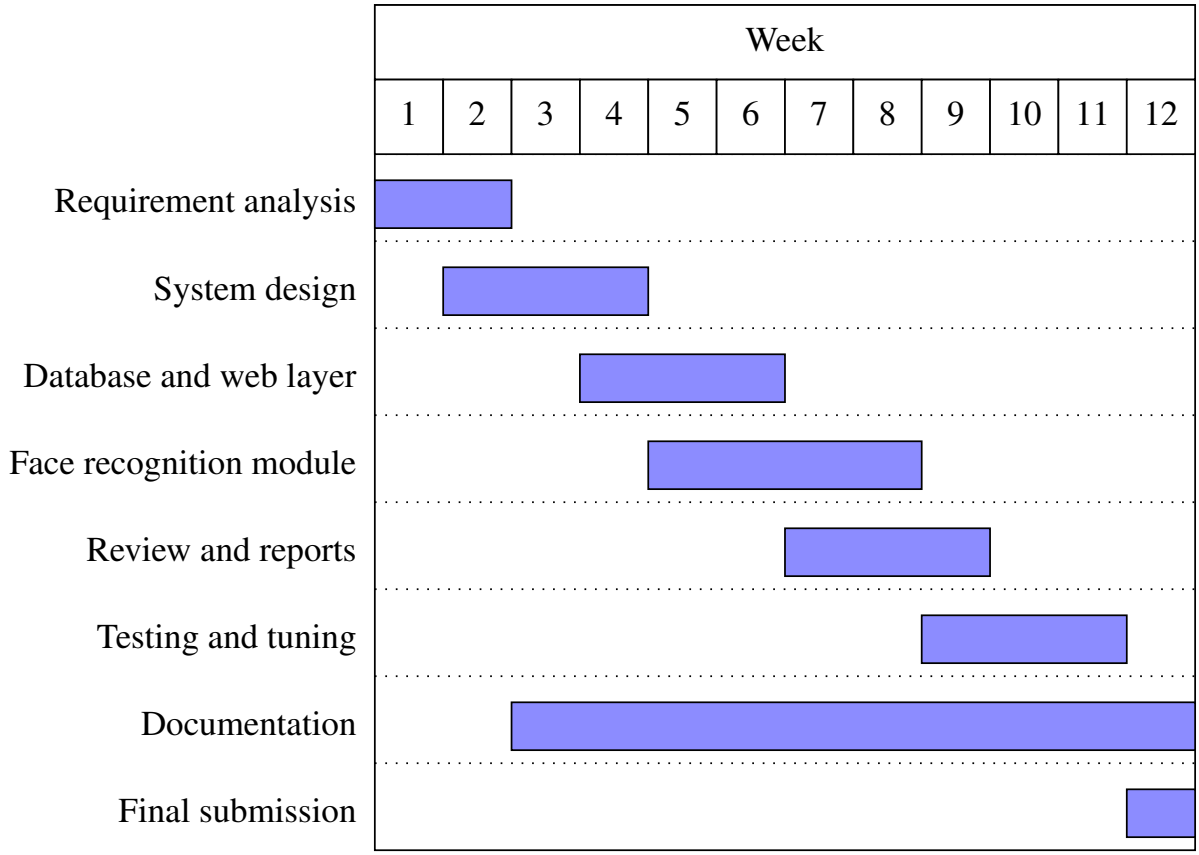


Figure 6.2: Gantt chart of the project work plan

possible matches and lowers the chance of a wrong identification. Using several photographs per student makes the match more robust to changes in pose, light and expression.

Figure 6.3 shows how a student is enrolled and Figure 6.4 shows the recognition pipeline used during a class.

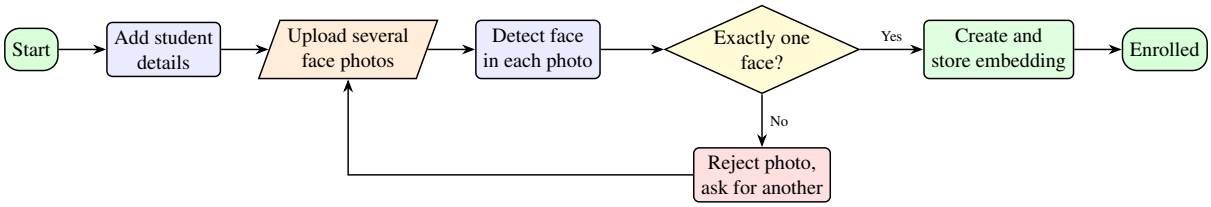


Figure 6.3: Enrolment of a student with face photographs

6.4 Database Design

The database has eight tables: Teacher, Student, StudentFace, Course, Enrollment, ClassSession, Attendance and UnknownFace. A student can have many face embeddings, can be enrolled in many courses, and has at most one attendance mark per class session. The relationships are shown in Figure 6.5.

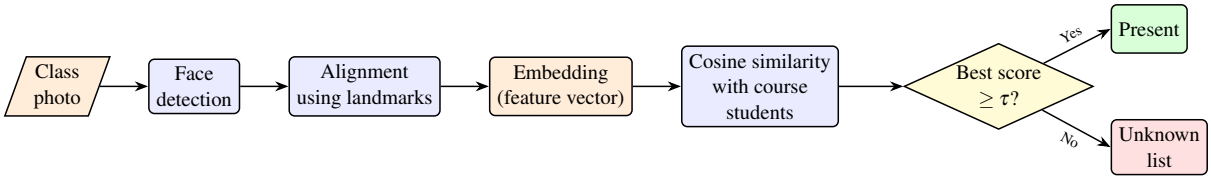


Figure 6.4: Face recognition pipeline used during a class

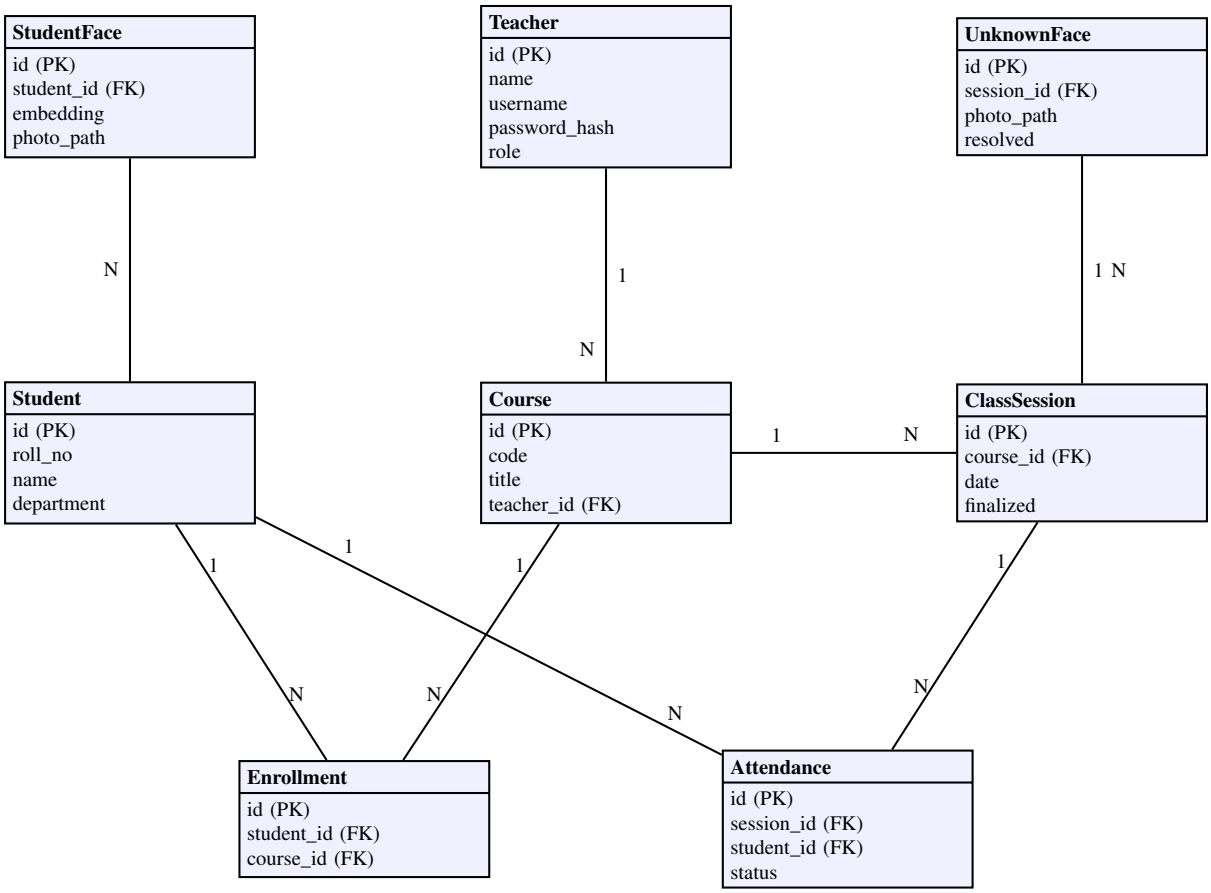


Figure 6.5: Entity-relationship diagram

6.4.1 Life Cycle of a Class Session

A class session passes through a few states, shown in Figure 6.6. A finalized session is locked until the teacher re-opens it, which protects the record from accidental changes.

6.5 Technologies Used

6.6 Security Measures

Passwords are stored only as hashes. Pages other than the login page require an authenticated session. Destructive actions such as deleting a student or a course are limited to the administrator, and a finalized class must be re-opened explicitly before its marks can be changed.

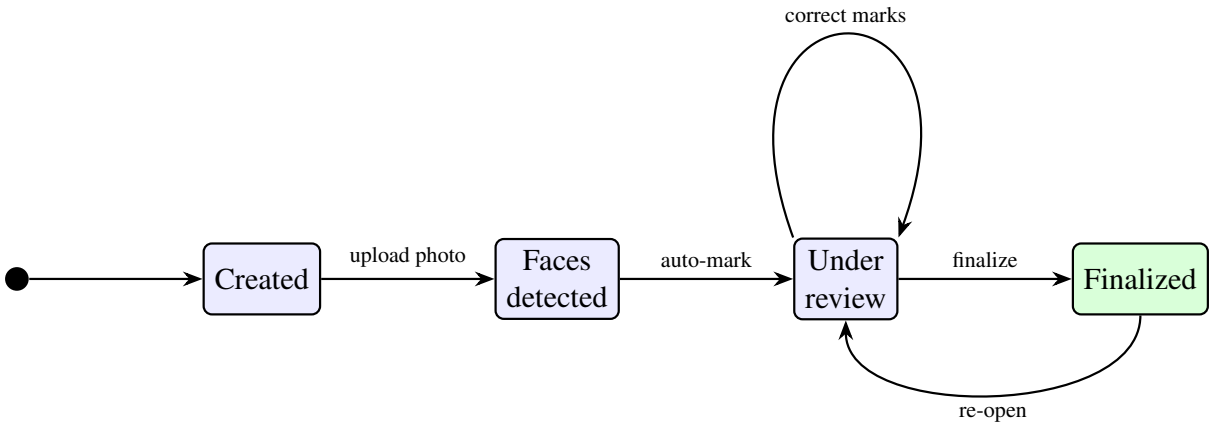


Figure 6.6: State diagram of a class session

Table 6.1: Technologies used in the project

Part	Technology
Back end	Python, Flask
Face detection and embedding	InsightFace
Image handling	OpenCV, NumPy
Database	SQLite
Front end	HTML, CSS, JavaScript
Reports	CSV export, printable month sheet

7. Requirements

7.1 Functional Requirements

- The system shall allow login by a teacher or an administrator.
- The system shall allow adding, editing and (for the administrator) deleting students, courses and teachers.
- The system shall store several face photographs and embeddings for each student.
- The system shall detect and recognise faces in a camera frame or uploaded photograph.
- The system shall let the teacher correct marks and review unknown faces.
- The system shall finalize a class and allow it to be re-opened.
- The system shall produce a session report, a month sheet and a CSV file.

7.2 Non-Functional Requirements

- **Usability:** a teacher should be able to take attendance in a few clicks.
- **Performance:** recognising a class photograph should take only a few seconds on an ordinary laptop.
- **Security:** hashed passwords and role-based permissions.
- **Portability:** works on any modern browser without installing software on the teacher's device.
- **Maintainability:** separate modules for the web layer, the face service and the database.

7.3 Hardware and Software Requirements

Table 7.1: Hardware and software requirements

Item	Requirement
Processor	Intel Core i5 or equivalent
Memory	8 GB RAM
Camera	Webcam or phone camera
Operating system	Windows 10/11 or Linux
Language	Python 3.9 or later
Browser	Chrome, Edge or Firefox

8. Expected Outcome

At the end of the project the department will have a working web application that:

- reduces the time spent on attendance to a few seconds per class;
- reduces proxy attendance because identity is checked from the face;
- keeps every record in one searchable database;
- produces reports and month sheets without manual calculation;
- keeps the teacher in control through review and correction of uncertain results.

8.1 Limitations

Recognition accuracy depends on light, camera quality, distance and the quality of the reference photographs. Students who hide their faces or sit far from the camera may be sent to the unknown list. For this reason the teacher review step is a deliberate part of the design.

8.2 Future Work

The system can be extended with liveness detection to resist photographs held in front of the camera, a mobile application, automatic notification to students with low attendance, and integration with the university result system.

9. Conclusion

The Smart Attendance System replaces manual roll calling with a fast, face-based process while keeping the teacher in control. It combines modern face recognition with a complete workflow of course management, review, roles and reports. The design is simple enough to run on a department computer and can be improved step by step in future work.

10. How to Run

1. Install Python 3.9 or later.
2. Open a terminal in the project folder and create a virtual environment.
3. Install the required packages from the requirements file.
4. Start the Flask server.
5. Open the address shown in the terminal in a web browser and log in.

Bibliography

[1] F. Schroff, D. Kalenichenko and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in *Proc. IEEE CVPR*, 2015, pp. 815–823.

[2] J. Deng, J. Guo, N. Xue and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” in *Proc. IEEE CVPR*, 2019, pp. 4690–4699.

[3] J. Deng, J. Guo, E. Ververas, I. Kotsia and S. Zafeiriou, “RetinaFace: Single-shot multi-level face localisation in the wild,” in *Proc. IEEE CVPR*, 2020, pp. 5203–5212.

[4] InsightFace Project, “InsightFace: 2D and 3D face analysis project,” GitHub repository. Available: `https://github.com/deepinsight/insightface`

[5] Pallets Projects, “Flask documentation.” Available: `https://flask.palletsprojects.com`

[6] SQLite Consortium, “SQLite documentation.” Available: `https://www.sqlite.org/docs.html`